CTI / v5

Clinical Trial Intelligence Workbench

Full operating & testing guide

Import. Predict. Review. Preserve.

Saved models, project access and traceable research evidence.

Windows / macOS / Linux - Python 3.12

A local research workbench with a real optional AI assistant.

PROTOCOL

EVIDENCE

SCENARIO

Research decision support - not clinical or regulatory advice

Includes first-owner setup, every-screen checks and model governance.

Static preview and full Python backend have different capabilities.

Manual edition: 9 October 2026 | Research use only

Contents

Use chapter bookmarks in your PDF viewer. The operating sequence, every-screen checks and research limits are all included.

01. What this release does . . . 3
02. Install and start on Windows . . . 4
03. First owner, login and projects . . . 5
04. Roles, invitations and limits . . . 6
05. Your first complete workflow . . . 7
06. Files, Excel, JSON and pasted data . . . 8
07. Registry, URL and AACT imports . . . 9
08. Schema and strict pretrial evidence . . . 10
09. Prediction Studio and saved evidence . . . 11
10. Model registry, approval and rollback . . . 12
11. Overview and Study Explorer . . . 13
12. Complexity Lab and Formula Studio . . . 14
13. Protocol Lab and human review . . . 15
14. Research Hub and similar trials . . . 16
15. Digital Twin and scenario planning . . . 17
16. Owner experiments and candidate training . . . 18
17. What the fitted models achieved . . . 19
18. Validation Lab and uncertainty . . . 20
19. Phase Academy and Phase Model . . . 21
20. Configure a real AI provider . . . 22
21. Saved contextual conversations . . . 23
22. Remote PostgreSQL and durability . . . 24
23. Backup, privacy and operating limits . . . 25
24. Troubleshooting and honest failures . . . 26
25. Every-screen acceptance and tests . . . 27
26. Reproduction and reference map . . . 28

01. What this release does

This is a research workbench for inspecting trial records, descriptive protocol complexity, saved duration and phase models, source evidence, and hypothetical scenarios. It preserves the v4 research tools in a separate v5 application. Keep your original v4 directory and its data unchanged; extract v5 into a new folder.

The normal v5 sequence is **import -> validate -> approved saved model -> prediction -> saved evidence**. Importing does not fit a model. Model training, research approval and activation are independent owner actions. The supplied release includes genuinely fitted, hash-pinned research artifacts, so ordinary compatible inference does not require retraining.

Capability	What to expect
Full local Python application	Session login, projects, imports, saved inference, protocol evidence/review, experiments, persisted conversations and settings
Browser preview	Interface demonstration and explicitly labelled frozen/fixture evidence; no live Python training, database, provider call or private upload
Scientific scope	Retrospective completed-trial duration, prolonged-duration scores, registry-phase consistency and descriptive formulas
Unavailable research outputs	Validated amendment probability, recruitment-risk probability, delay-versus-original-plan probability and clinical-success prediction
Integrations requiring your configuration	Real AI provider account/key/model, remote PostgreSQL, and a reviewed production hosting environment

The authentic starter contains 800 public registry records. Only a subset qualifies for each experiment. The duration test set has 64 trials; the phase test set has 59. Added models do not establish a large or clinically validated benchmark. Latest-state registry narratives may contain information written after trial start.

Research approval is permission to use a software artifact for research. It is not clinical, regulatory or prospective validation. Keep software tests, held-out metrics and external clinical validation distinct.

This manual follows the v5 code, configurations and fitted artifacts. The v4 guide remains an archive of the previous application. Use `V5_MODEL_CARD.md`, `V5_PRETRAINED.md`, `V5_PLATFORM.md` and `V5_SCIENTIFIC_BOUNDARIES.md` for deeper implementation contracts. Final release test evidence belongs with the release verification files; a manual instruction is not evidence that your machine or external services have been tested.

Acceptance check: identify whether you are viewing the static preview or running `backend.app.main:app`. A preview must not be used to conclude that authentication, saving or provider billing works.

02. Install and start on Windows

Install **Python 3.12** with the Python launcher. Extract the release into a writable new v5 folder, for example `C:\CTI\clinical-trial-intelligence-workbench-v5`. Do not run from inside the ZIP or overwrite v4. The included built interface means Node.js is unnecessary for normal use.

1. Double-click `SETUP_WINDOWS.bat`. It runs the Python 3.12 setup script, creates `.venv`, installs dependencies and creates `.env` if needed. Keep the terminal open until setup completes.
2. Open `.env` in a text editor. Setup generates a private `SETUP_TOKEN` if it is absent or blank; it does not print the token. Copy only its value when the first-owner form asks for it. Preserve configured settings when rerunning setup.
3. Double-click `START_WINDOWS.bat`. Keep this server terminal open. Open `http://127.0.0.1:8000` in your browser.
4. Complete first-owner setup as described next. The app and API are served from the same local address.
5. Run `CHECK_WINDOWS.bat` if setup or startup fails. It checks the interpreter, modules, built UI, starter file and storage, then runs `pip check`. It does not test paid AI access or remote database persistence.

Equivalent commands in **Command Prompt**, from the extracted root:

```
py -3.12 scripts\setup.py
.venv\Scripts\python.exe scripts\doctor.py
.venv\Scripts\python.exe scripts\start.py
```

If the prebuilt UI is missing or you intentionally changed frontend source, install Node.js 22, then run:

```
cd frontend
npm ci
npm run build
cd ..
.venv\Scripts\python.exe scripts\start.py
```

Setup needs access to package downloads. Once installed, the bundled data and saved-model workflow can run without an AI key. New registry imports and AI replies need network access. Use the `.venv` interpreter for all application commands; a different global Python can have incompatible dependencies.

On macOS/Linux, use `python3.12 scripts/setup.py` and `.venv/bin/python scripts/start.py`. Stop the application with **Ctrl+C**. A cancellation traceback immediately after an intentional shutdown can be harmless; see chapter 24.

Acceptance check: the terminal reports the browser URL, the login/setup page opens, and the diagnostics report Python 3.12 and a writable storage directory.

03. First owner, login and projects

The default mode is `AUTH_MODE=session`. There is no public administrator registration. A first owner claims a new installation using the private setup token from the server `.env` file. Use an email address, display name and a password of at least 12 characters. The setup claim is transactional and works only while no owner exists.

```
APP_ENV=development
AUTH_MODE=session
API_HOST=127.0.0.1
API_PORT=8000
STORAGE_DIR=./data/workbench
SESSION_HOURS=12
```

The generated `SETUP_TOKEN` line is also required for the first claim. Do not paste a token, password or provider key into a screenshot, chat prompt, source-control commit or frontend setting. After the first owner exists, sign in with that account; rerunning setup is not a password reset.

Passwords are salted and hashed with scrypt. The browser receives a random HttpOnly session cookie, and the metadata database retains a digest rather than the raw session token. Session expiry defaults to 12 hours; the server allows 1-168 hours. Logout revokes the server session. Production cookies require HTTPS.

Open Project Settings to create or select a project. Project identity travels with research requests, and the server checks membership before reading datasets, experiments, scenarios, exports and conversations. Changing the selected project must also change the visible data and conversation context. A project name is a label; generated project IDs determine storage and access boundaries.

Situation	Correct action
New installation shows setup	Use the token from this installation's `.env`
Existing installation shows login	Use your existing account or a valid invitation
Login expires	Sign in again; do not delete storage
Need another project	Create it in Project Settings, then import its data
Want an offline identity-free workspace	Explicitly set `AUTH_MODE=local` only for local development

Local mode is a single development workspace without account security. It cannot start as production and must not be exposed to the internet. The legacy `API_BEARER_TOKEN` is not a substitute credential for v5 sessions.

Acceptance check: create two projects, import different records, switch between them, and confirm datasets and saved history stay separated. Logout, then verify a private request requires login again.

04. Roles, invitations and limits

Project owners manage membership. In Project Settings, open Members, enter the recipient's email and role, then save. An existing account receives membership. A new account receives a one-time invitation token that expires after 24 hours. Share it privately with the intended person; **the application does not send email**. The recipient uses Accept invitation and chooses their own name and password.

Role	Main permissions
Owner	Read/predict, import, train candidates, approve/activate models, review protocols, call AI, manage settings/members and inspect audit
Researcher	Read/predict, import records/protocols, save scenarios and use their own AI conversations; cannot train or promote
Reviewer	Read research evidence, review protocols and inspect audit; cannot import, train, promote or request paid AI replies
Viewer	Read permitted project evidence; cannot import, train, promote or request paid AI replies

The backend enforces permissions, including older `/api/v1` routes. Hiding a button is not the security boundary. Conversations belong to a user within a project; being another project owner does not grant access to someone else's private chat through these APIs.

Project Settings > Usage & limits exposes the actual stored settings and usage. Owners choose default provider/profile, daily assistant limit and retention. The default assistant limit is 50 requests per project per UTC day; 0 pauses AI replies, and the supported range is 0-500. Retention is 1-3650 days, default 90. Inactive old conversations expire when chat routes are accessed; this is lazy retention, not a scheduled deletion service.

Server defaults include 25 MB uploads, 100,000 dataset rows, 20,000 training rows, 30 projects per account, 200 conversations per user/project and 200 messages per conversation. Some formats impose tighter limits. Training allows one active job across projects in the process. Settings may lower or raise bounded defaults, but must be reviewed with memory and operating costs in mind.

AI quotas count assistant requests, not dollars or output tokens. A configured-provider failure consumes a reserved request, and eligible fallback can call more than one provider within it. An unavailable/unconfigured provider does not consume quota.

Acceptance check: use a researcher/viewer account and attempt a forbidden training/import operation. Confirm denial creates no new run or dataset. Shortening retention can remove history; test it only in a disposable project.

05. Your first complete workflow

Use this sequence to establish the application works before importing your own files or configuring an AI provider.

1. Sign in and choose the intended project. Open Data Manager and select **Load starter cohort**. This copies the bundled authentic 800-record source into the project's registry. It does not download a new sample or start training.
2. Inspect the dataset name, count, missingness and source checksum. The small interface fixture is a software test source; choose the authentic starter for scientific model output.
3. Open Prediction Studio. Choose **Duration + prolonged-duration risk**, **Active approved model**, and **Retrospective research screening**. Run Predict now if the import flow has not already initiated the saved-model run.
4. Select a result. Review duration, raw prolonged-duration probability, uncertainty, source/model provenance, missing features and support warnings. The supplied duration champion is a median baseline; a constant output across records can be correct.
5. Export Evidence JSON and optionally CSV. Reopen the saved prediction run after a browser refresh. The JSON preserves the inference-time model and input hashes.
6. Open Study Explorer and inspect a record. Use Research Hub for connected model evidence and similar trials, Complexity Lab for descriptive PCI, and Digital Twin for a clearly hypothetical scenario.
7. Change the inference mode to **Strict pretrial availability** and repeat. Latest-state starter records lack feature timing evidence, so blocked records are expected. Do not add invented timestamps to make them pass.

Output	How to interpret it
Available	The saved artifact produced a compatible finite output; research limitations still apply
Blocked	Strict pretrial evidence checks failed; prediction is null
Unavailable	No compatible approved artifact/output is available; do not substitute a made-up number
Warning	Read the reason; a warning can expose missingness, distribution limits or retrospective scope

The trial journey is an explanation of stages. Its speed and stage widths do not predict activation, recruitment, treatment or follow-up durations. Only the labelled total from the actual model is a model result.

You do not need to run Experiments to complete this workflow. Training is for an owner who deliberately wants a new candidate. An AI provider is optional; importing, formulas, saved predictions and evidence review remain useful without it.

Acceptance check: note the active model hash before import and after prediction. It should remain unchanged, with no new experiment created. Save a batch, restart the server, and reopen that batch in the same project.

06. Files, Excel, JSON and pasted data

Data Manager imports canonical **CSV, Parquet, flat JSON and** `.xlsx` files. Start with Download CSV template to obtain field names. Give a useful dataset name, choose the file, then Validate & import. A failed validation must leave no committed partial dataset. Source bytes, canonical records and validation/provenance information are preserved in project storage.

Method	Input contract
CSV	Header row; unique column names; one scalar record per row; quote commas/newlines inside text
JSON	Nonempty array of flat objects; unique keys; finite numbers; no nested structures
Excel `.xlsx`	First worksheet only; unique nonempty headers; values in rows; formulas are not evaluated
Parquet	Canonical tabular fields matching the same schema
Paste	Select CSV or JSON and paste canonical records, up to 2 MB or the server upload cap

Excel reads cached formula values only. If cached results are missing, save a values-only workbook before import. Legacy `.xls`, `.xlsm` macros, encrypted workbooks and binary spreadsheets are not supported. The importer also bounds archive expansion, rows, columns and cells, and rejects unsafe XML constructs. These protections are not a malware scanning service.

Every record requires nonempty `nct_id` and `title`. IDs must be unique within a dataset. Model inference additionally needs useful structured features or narrative fields for the selected model modality. A valid two-column registry is not necessarily a model-compatible input.

A minimal paste example below is a **synthetic format example**, not clinical evidence or a training cohort:

```
nct_id,title,phase,enrollment,number_of_arms,primary_outcomes
FORMAT-DEMO-001,Format demonstration,PHASE2,120,2,1
```

Do not mix sources without keeping their provenance, dates and meaning. Do not turn an absent count into zero merely to satisfy a model. Preserve exact date precision and the distinction between planned/estimated and actual completion.

Paste import and protocol-text import are different workflows. Paste data creates tabular study rows. Protocol Lab stores source document text, reviewable excerpts and extracted candidates; it does not automatically create a validated protocol-prediction dataset.

Acceptance check: import a small genuine CSV, then equivalent JSON/XLSX; compare IDs and values. Submit a duplicate ID, negative count and nested JSON value in a disposable test and verify a clear validation failure. Confirm the active approved model never changes.

07. Registry, URL and AACT imports

The ClinicalTrials.gov panel accepts a registry search expression and a maximum count of 50, 100, 250, 500 or 1,000. It uses the dedicated API v2 mapper and records the query and raw response. New downloads are changing convenience samples; they are not exact reproductions of the frozen starter. A trial's current registry state does not prove its fields existed before trial start.

The separate HTTPS URL importer accepts a direct public CSV or flat-JSON file. Default exact hosts are `clinicaltrials.gov` and `raw.githubusercontent.com`; the operator can deliberately configure `IMPORT_ALLOWED_HOSTS`. A normal web page, nested API response or share page is not a canonical data file.

URLs must use HTTPS with no embedded credentials, query string, fragment, nonstandard port or IP literal. Private/internal destinations and redirects are rejected. Do not put signed links or tokens in dataset URLs. The dedicated registry importer is the correct route for nested ClinicalTrials.gov API data.

AACT is an **offline frozen-table workflow**, not a provisioned live AACT database connection. Obtain the selected snapshot and an independently recorded expected checksum, keep raw files unchanged, then verify and transform them with the scripts. The extractor supports headered UTF-8 pipe-delimited `.txt` or comma-delimited `.csv` AACT tables. Inspect its missing optional table report.

```
.venv\Scripts\python.exe scripts\verify_aact.py --path data\raw\aact_snapshot --output
  data\aact-verification.json
.venv\Scripts\python.exe scripts\build_features.py --input data\raw\aact_snapshot --output
  data\processed\aact-v5 --snapshot-date YYYY-MM-DD
```

Replace `YYYY-MM-DD` with the real frozen snapshot date. Upload the resulting `trials.csv` through Data Manager and retain `source_metadata.json`, the verification manifest and raw snapshot alongside the project evidence. A browser upload does not automatically ingest an adjacent metadata sidecar; use the sidecar explicitly when reproducing CLI research.

`download_aact.py` can download a chosen HTTPS snapshot with `--url`, `--sha256`, `--snapshot-date` and `--output`. It refuses mismatched checksums or an existing output. It requires your legitimate source access; it does not discover credentials or bypass an AACT login.

Acceptance check: inspect source counts/checksums after any registry import. For AACT, rerun verification with `--expected` pointing to the saved verification manifest and confirm raw bytes match. A failed network acquisition must not be reported as a completed dataset.

08. Schema and strict pretrial evidence

The canonical schema preserves study identity, descriptive features and available provenance. Missing values remain missing. Numeric feature columns accept finite nonnegative numbers. Date parsing preserves partial precision; day-precision actual dates are required by the completed-duration training endpoint.

Field group	Examples and meaning
Required identity	`nct_id`, `title`
Structured inputs	`enrollment`, `number_of_arms`, `primary_outcomes`, `secondary_outcomes`, `criteria_count`
Categories	`study_type`, `phase`; inspect the selected model's feature list
Narrative inputs	`brief_summary`, `detailed_description`, `eligibility_criteria`, and title
Endpoint/source fields	`start_date`, `completion_date`, `completion_date_type`, `status`; these are not permission to leak outcomes into features
Strict inference timing	`prediction_as_of`, exact `start_date`, and `<feature>_available_at` for each observed consumed feature

Strict pretrial mode requires a valid prediction timestamp no later than trial start and an exact start date. Each observed feature consumed by that model needs a valid availability timestamp at or before prediction time. Examples include `enrollment_available_at`, `title_available_at` and `phase_available_at` when those inputs are consumed. The model manifest determines which fields are checked; duration and phase tasks use different feature sets.

Unknown or later timestamps block that row and return `prediction:null`. The checks concern supplied timestamps; the application does not independently certify that they are historically true. Some missing values can use saved training imputers, but absent model modalities fail schema validation. Inspect warnings and feature coverage rather than treating an accepted row as complete evidence.

Actual final enrollment, future amendments, completed outcomes and later narrative revisions must not be passed off as design-time values. A snapshot download date, old trial start date or global baseline flag cannot establish feature-level availability. Use retained historical evidence or keep the work explicitly retrospective.

Protocol Lab also offers a timestamp audit utility. Passing that separate utility does not attach verified provenance to every prediction automatically and does not clinically validate a model.

Acceptance check: a latest-state starter batch should block in strict mode. In controlled software tests, an unknown timestamp and a timestamp later than prediction time must also block. Only authentic historical source evidence can justify real supplied timestamps.

09. Prediction Studio and saved evidence

Prediction Studio applies the approved persisted pipeline to the selected project's dataset. Choose Duration + prolonged-duration risk or Phase consistency, then an active or explicitly approved saved model. Select retrospective or strict pretrial mode. The interface requests the first 100 validated rows; the API permits 1-1,000 per batch. This is a bounded batch view, not an assertion that every row of a large upload was scored.

During inference, preprocessing and model parameters come from the saved artifact. No fit, calibration or model-selection step runs on your uploaded data. A missing or incompatible model fails explicitly instead of fitting a replacement. New data need not have the training dataset's identity or checksum.

For duration, read all of the following together: point estimate in days, raw prolonged-duration probability, model-specific prolonged threshold, uncertainty status, missing feature warnings, distribution/support warnings and artifact provenance. Prolonged means exceeding a training-derived duration threshold. It does not mean being late against an original plan.

For the bundled release, minimum validation duration MAE selects the training-median baseline. Its predictions can be constant across patients or studies. This is a cohort baseline estimate, not an individualized biological forecast. The paired risk probability comes from that same selected branch; it is not automatically the branch with lowest risk Brier score.

Control or evidence	Behavior
Records list	Shows available, blocked or unavailable result status per study
Why this prediction?	Exposes actual explanation, support and calibration metadata
Input and model provenance	Separates incoming dataset identity/hash from artifact training provenance
Evidence JSON	Exports the persisted batch with its model, dataset, warnings and timestamp
CSV	Compact tabular output; retain JSON for complete evidence
Print / PDF	Uses browser printing; inspect the print preview before saving
Saved prediction runs	Reopens prior persisted batches without recomputing their model references

Null or unbounded interval values must remain explicitly unavailable/unbounded. A broad interval is a result to interpret, not a formatting error. Tree-global or missing nonlinear attribution must not be called local SHAP.

Acceptance check: refresh and reopen a saved run, then activate another approved model in a test project. The earlier saved run must retain the exact model/hash that generated it. Exported evidence should match the onscreen run.

10. Model registry, approval and rollback

Model Registry separates **candidate**, **approved**, **active**, **ineligible** and **unavailable** states. The server release catalog `examples/approved-models-v5.json` pins bundled manifest and executable artifact hashes. A uploaded data file cannot become an executable model, and client-supplied joblib paths are unsupported.

1. The owner imports a suitable dataset and runs a deliberate duration or phase experiment. The old approved model remains available while that candidate trains.
2. After completion, inspect the manifest, training source, split sizes, task metrics, missingness, temporal limits and intended use. Its registry ID is `project-<experiment-id>`.
3. Enter a meaningful rationale in Model Registry and select **Approve for research**. Approval checks compatibility, source identity/hash, input/pipeline hashes, nonfixture provenance and real finite validation/test task metrics with held-out rows.
4. Select **Activate / roll back** separately. Approval alone does not change the active pointer.
5. To roll back, activate a previously approved model. Existing saved evidence keeps its old model identity.

The interface requires at least ten characters of rationale. The server records owner identity, time, notes and exact approved digests. Activation is persisted and audited. Keep the prior approved artifacts available for rollback and evidence reproduction.

Approval gates establish reproducibility and operational compatibility. They do not impose a clinical performance threshold or guarantee improvement. Undefined optional metrics can remain null when, for example, a split has only one risk class. Read reasons for ineligibility rather than editing the manifest to bypass a gate.

Failure	Expected response
Candidate still queued/running/failed	Cannot become a serving approved model
Training fixture or mismatched dataset hash	Approval gate rejects it
Artifact or manifest changed after approval	Checksum conflict; serving fails closed
Missing artifact/dependency incompatibility	Visible unavailable state; no automatic retraining
Researcher/reviewer/viewer requests promotion	Server denies the owner-only action

Do not overwrite approved bytes, hand-edit the release catalog or change a candidate's scores to make it eligible. Train a new version in a new output folder, review its evidence, then approve deliberately.

Acceptance check: train one disposable candidate, confirm it is not active, approve without activating, then activate and roll back. Verify each action and prior evidence identity in the audit/history. This is an operational lifecycle test, not scientific validation.

11. Overview and Study Explorer

Overview summarizes the active dataset: record counts, reported phases/statuses and descriptive complexity. Treat these summaries as source descriptions. They are not trial success rates, evidence of clinical benefit or population-representative estimates.

Use Study Explorer to search records, filter reported phase/status and sort by identifier, title or enrollment. Pagination requests 25 rows at a time. Opening a record shows source fields, descriptive PCI, provenance and available model evidence. Some connected selectors contain the first 100 records plus studies you opened in Explorer; open a study here if it is missing from a downstream selector.

1. Select the intended project and dataset first.
2. Search by study ID or title, inspect phase and status filters, and open a detail panel.
3. Compare selected studies across source title, phase, status, enrollment, arms, outcomes, criteria and PCI. Prefer small, interpretable comparisons.
4. Export CSV when needed. Record which project/dataset and filters produced the view, and keep a full provenance export for reproducibility.

An identifier matches a source record; it does not imply that the record is complete or valid at pretrial time. Missing enrollment, absent narratives and estimated dates can change what downstream models can support. Check Data Manager missingness and the actual detail fields before interpreting a prediction.

The same trial can appear in different imported snapshots. Keep dataset IDs/hashes distinct. Do not compare their outputs as if only the model changed when the source records changed too. A systematic model comparison holds cohort, endpoint and split fixed.

Question	Best next screen
What source record am I inspecting?	Study Explorer / Data Manager
Why does this descriptive score change?	Complexity Lab / Formula Studio
What saved model produced this output?	Prediction Studio / Model Registry
Which historical records share text?	Research Hub
What changes under an explicit assumption?	Digital Twin
Was the model actually evaluated?	Experiments / Validation Lab / model card

Acceptance check: search a known ID, change a filter, move to the next page and reopen details. Change projects and confirm no previous project record remains selected. Exports should contain the intended data, with no password, provider key or server connection string.

12. Complexity Lab and Formula Studio

The fixed-reference PCI is a transparent descriptive score from 0 to 100. It is separate from the training-relative PCI feature and the seven-domain protocol rubric. Its caps and weights are explicit research assumptions, not clinically validated thresholds.

Component	Field	Ceiling	Weight
Eligibility burden	`criteria_count`	40	25
Primary outcome burden	`primary_outcomes`	10	20
Secondary outcome burden	`secondary_outcomes`	20	20
Arm complexity	`number_of_arms`	8	15
Enrollment burden	`enrollment`	1000	10
Text complexity	`text_complexity`	20	10

```
PCI = sum(weight * clip(value / ceiling, 0, 1))
Low: score < 40; Medium: 40 through 70; High: score > 70
```

The total is rounded once to two decimal places using exact decimal half-up semantics, and the rounded value determines the category. Missing, invalid and nonfinite components contribute zero and are disclosed. This can understate an incomplete record: a low PCI is not proof of low actual burden. Negative finite formula inputs clip to zero; canonical dataset validation rejects negative count values.

Complexity Lab supports reference weights, equal weights and normalized custom sensitivity weights. Enrollment adjustment creates a hypothetical local score; it does not modify the source or retrain a model. Export the scenario if you need a record of the assumptions. Custom weights change the index definition, so label them when comparing results.

Formula Studio exposes each value, cap, weight and contribution. Play, pause, step or reset the terms. Reduced-motion preference disables automatic term playback and preserves manual stepping. Its known-answer metric, cosine, planning and conformal-rank examples test arithmetic, not clinical performance.

When every component equals half its cap, the total is 50. When every component reaches its cap, the total is 100. A record with all components missing yields zero with all missing fields disclosed, which must not be interpreted as a complete low-complexity protocol.

Acceptance check: step through all six terms and compare the final score with Complexity Lab. Test half-ceiling values and rounding boundaries. For metric definitions and exact test inputs, read `configs/formulas.json` and the formula tests; illustrative UI examples may use different small vectors.

13. Protocol Lab and human review

Protocol Lab stores document evidence separately from trial datasets. Owners and researchers can paste text or upload a text-based PDF, UTF-8 TXT or Markdown file. The document limit is 8 MB, 150 PDF pages and 250,000 extracted characters. Encrypted PDFs are unsupported. Scans need OCR outside the app before import; no OCR accuracy is claimed here.

Give each protocol a title. For a revision, select the existing version family so the new document remains linked to the earlier version. Original bytes are preserved, with source and extracted-text hashes. Extracted evidence retains page, section, quote, method and a review-required flag.

The current parser uses deterministic rules to propose phase/enrollment values and keyword evidence in seven domains. Multiple conflicting mentions may leave a field unknown. A phase mention can describe a prior study. Keyword counts are evidence locators, not measured patient burden or validated semantic extraction.

Review feature	Meaning
Pending	No human review decision yet
Accepted	Owner/reviewer accepted the evidence with notes
Corrected	Owner/reviewer supplied supported field corrections and notes
Rejected	Evidence was rejected; it is excluded from assistant document retrieval
Version comparison	Added/removed text lines and source hashes; no amendment probability

Owners/reviewers may correct phase, enrollment, arm count, primary/secondary outcome count and criteria count. Corrections preserve original features and record actor/time/status/notes. They do not retroactively rewrite original document bytes or silently retrain models. Review acknowledgement is not a qualified electronic signature.

The seven-domain rubric covers design, eligibility, patient, site, endpoint, geography/regulatory and intervention/safety burden. Users supply reviewed 0-100 scores and finite nonnegative weights. If any domain is unknown, the composite remains null. With all domains present, it computes weighted mean; the rubric is exploratory and not clinically validated.

There is no bundled protocol-PDF-specific predictive model. Registry-model test performance does not validate PDF extraction or make an extracted protocol ready for clinical forecasting. Keep extraction review, timestamp auditing and model inference evidence distinct.

Acceptance check: import two short public/synthetic versions, inspect hashes and source quotes, compare text, then record a corrected review as owner/reviewer. A researcher should be denied review. A missing domain must keep the seven-domain total unknown; comparison must never invent amendment risk.

14. Research Hub and similar trials

Research Hub brings source details, descriptive complexity, duration/phase evidence and retrieval into one study-centered view. Select the source study and inspect the provenance before moving to a scenario or report. An unavailable model output should retain its reason while other descriptive panels remain usable.

Similar-trial retrieval uses TF-IDF text overlap with optional exact phase filtering. It excludes the source study from its own matches. Read matching explanation, score and pool information rather than treating a high text score as clinical equivalence. Shared boilerplate can increase similarity without establishing comparable populations, interventions or endpoints.

Evidence	Supports	Does not establish
Registry source fields	What this imported snapshot reports	What was known at protocol finalization
Fixed-reference PCI	An explicit descriptive weighted calculation	Clinical risk or trial success
Saved duration output	This artifact's retrospective estimate	A causal effect or individualized clinical prediction
TF-IDF cosine match	Overlap in weighted text terms	A matched external validation cohort
Phase consistency output	Model agreement with registry-style phase patterns	A recommended regulatory phase
Source/model hashes	The identity of recorded bytes	Truth of source facts or clinical validity

Exports are evidence packets for review. Preserve their dataset/model identifiers, limitations and support warnings. A compact screen summary omits detail, so retain full JSON alongside any screenshot or printed report. Hashes help detect changed content, but do not turn records into independently immutable regulated archives.

The optional frozen biomedical-embedding script is not the active default retrieval system or approved model input. It requires a pinned model revision and actual weights. Do not infer a working dense biomedical index, ontology normalization or molecular model from a module being present in the repository.

Protocol document context used by chat is also bounded lexical retrieval. It selects matching excerpts from recent project documents, records source hashes/review states, and excludes rejected documents. A supplied excerpt reference shows what context was offered to the provider; it is not proof that every generated sentence is supported.

Acceptance check: select a study with narrative text and confirm it does not retrieve itself. Change the phase filter and inspect the candidate pool. Open a matched source record and compare actual fields. Export a packet and confirm its IDs/hashes correspond to the current study and dataset.

15. Digital Twin and scenario planning

Digital Twin compares a preserved source record with an explicitly hypothetical design. It saves scenario inputs, computed outputs, assumptions, model provenance and an evidence checksum. Editing a form after saving does not rewrite an existing saved result.

1. Select a source trial. Read its original features and model availability.
2. Give the scenario a name. Change only supported design fields and keep the new combination scientifically plausible.
3. Enter planning assumptions: target enrollment, sites, monthly recruitment rate per site, startup months and follow-up months. Defaults are editable assumptions, not observed registry facts.
4. Choose Compute & save scenario. Compare fixed-reference PCI, any compatible fitted duration output and raw prolonged-duration score.
5. Review warnings and unchanged source fields. Export Scenario JSON or reopen it from the source study's saved history.

```
Recruitment months = target / (sites * monthly rate per site)
Planning total = startup + recruitment + follow-up
Example: 240 / (4 * 5) = 12 recruitment months
Total: 3 + 12 + 6 = 21 months
```

This arithmetic assumes constant rates, simultaneously available sites and no attrition or ramp-up. It is not a learned recruitment forecast. The fitted duration model does not consume every planning control; changing an unused planning assumption affects only the separate planning calculation.

The saved duration champion may be a constant baseline, so changing source features can leave the fitted estimate unchanged. That behavior is consistent with the artifact. Do not substitute the planning total to make the prediction appear responsive.

Source-versus-scenario deltas are model associations. They do not prove that changing eligibility or enrollment will shorten a real trial. Correlated field changes can create cases outside training support. Read uncertainty and support metadata in the saved JSON.

The planning animation explains assumptions; playback speed does not simulate elapsed trial time or predict treatment effects. Use pause, reset and step controls where available, and respect operating-system reduced-motion preferences.

Acceptance check: calculate the 21-month example above, save it, then alter inputs without saving and confirm the recorded result stays unchanged. Restart the server and reopen the scenario. Check original source records have not changed. In a preview, scenario history may be browser-memory-only and must be labelled accordingly.

16. Owner experiments and candidate training

Experiments is an intentional owner workflow. It produces candidates in new artifact folders; it does not replace the approved serving model. Researchers can inspect evidence but cannot start training through either the UI or direct legacy API routes.

Load a real dataset, inspect quality and training preflight, then select Run experiment. The completed-duration cohort requires appropriate completed records, known actual completion and start dates with supported precision, positive duration and adequate retained temporal splits. Small or badly dated datasets may import successfully yet fail training readiness.

Duration means actual study completion minus start in days. Prolonged duration means exceeding the training cohort's 75th percentile. The bundled threshold is 1,101 days. This is a global training-cohort threshold, not a matched phase/indication threshold or delay against original plan.

The nine branches comprise median/prevalence baseline; structured, text and multimodal linear models; structured Extra Trees, Random Forest, conventional Gradient Boosting and Histogram Gradient Boosting; and a text/histogram late-fusion branch. More complex branches are comparisons, not assumed winners.

Partition	Permitted use
Training	Fit imputers/scalers/categories/vocabulary, PCI ranges and models
Tuning	Choose supported parameters, fusion weights and recommendations
Calibration	Fit supported probability transform and residual interval quantities
Untouched test	Describe final fixed-model performance; never select a champion

The pipeline groups start dates chronologically and purges outcomes unavailable by relevant cutoffs. It does not certify that latest-state narrative features existed at those cutoffs. Candidate approval requires source/config/hash integrity; scientific acceptance still needs expert review.

A single process permits one training job at a time. Native numerical thread limits reduce resource contention. A server restart marks queued/running jobs failed; there is no distributed durable queue. Start a new run deliberately after resolving the issue, without deleting completed artifacts.

Inspect manifest, split IDs, metrics, held-out predictions, diagnostics and warnings. The experiment's risk recommendation and duration recommendation can differ from the registry's duration-MAE serving selection. See chapter 17 before drawing conclusions from a small test result.

Acceptance check: use an owner to run one disposable experiment. Verify queued/running/completed states and real artifacts. Confirm the active saved model remains unchanged until separate approval and activation. A fixture must not qualify for scientific candidate approval.

17. What the fitted models achieved

The bundled artifacts were genuinely fitted on the frozen 800-record ClinicalTrials.gov starter retrieved on 7 October 2026. Duration eligibility retained 312 trials, with 488 excluded; temporal outcome-availability purging removed 123 more. Final splits contain **105 train, 9 tune, 11 calibration and 64 test** trials. The test has six prolonged outcomes.

Duration branch	Test MAE, days	Test RMSE, days	Test Brier
Median / prevalence baseline	351.19	419.23	0.10864
Structured linear	372.68	432.38	0.11959
Text linear	314.70	366.98	0.07757
Multimodal linear	333.00	399.28	0.09341
Extra Trees	354.22	418.79	0.10882
Random Forest	350.55	428.37	0.11279
Gradient Boosting	372.39	491.10	0.13411
Histogram Gradient Boosting	447.14	589.99	0.16266
Text + Histogram Fusion	317.13	374.88	0.08771

Lower errors and Brier are better. These test scores do not select the serving branch. Minimum validation duration MAE selects `median_baseline`; the separate risk-Brier recommendation is text. New histogram boosting performs worse here and is not presented as an accuracy gain.

Text-model test AUROC is 0.8075 with a 95% paired per-trial bootstrap interval of 0.5976-0.9626. Its MAE difference versus baseline is -36.49 days, interval -93.52 to 23.50. That interval includes zero. Small counts, single-class tuning/calibration and possible post-start text leakage sharply limit interpretation.

The phase cohort contains 291 eligible single-phase labels: 175 train, 57 tuning and 59 test. Logistic test accuracy is 0.5593, macro-F1 0.4752 and balanced accuracy 0.4818; majority-baseline accuracy is 0.3729. These are registry-label consistency results, not clinical success prediction.

Two larger acquisition attempts preserved overlapping partial downloads. Recovery yielded 1,800 records, only 156 with exact start/actual-completion pairs. Across starter and recovery there are 2,580 unique IDs. **The recovered partial source was not used for approved model training.** No massive-training claim follows from raw inventory counts.

Acceptance check: compare this table with `examples/starter-duration-v5/metrics.json`, the phase metrics and `V5_MODEL_CARD.md`. Review actual held-out predictions and cohort exclusions before reporting a metric.

18. Validation Lab and uncertainty

Validation Lab and experiment diagnostics expose the distinction between fitting, tuning, calibration and held-out evaluation. Select the intended dataset/run/model and split. Empty evidence can simply mean no project-local experiment exists; bundled saved inference remains a separate workflow.

Measure	Interpretation
MAE	Average absolute duration error in days
RMSE	Duration error that gives larger mistakes more weight
R-squared	Relative squared-error measure; can be negative; undefined for insufficient/constant targets
AUROC	Risk-score discrimination; undefined for a one-class split
Average precision	Ranking performance affected by event prevalence
Brier	Mean squared probability error; inspect raw versus calibrated status
Reliability bins	Predicted score versus observed fraction in each supported bin
Interval coverage/width	Observed held-out behavior of the stated interval method

Primary probabilities remain raw unless the separate calibration status explicitly says otherwise. Sigmoid calibration needs adequate reserved rows and class support. The starter's calibration cohort has no prolonged positive outcomes, so a probability transform cannot be meaningfully fitted there. Calibration-fitting diagnostics are not independent test validation.

Duration intervals use split-conformal absolute residuals. Finite-sample rank can exceed calibration size; in that case the valid interval is unbounded, represented without invented finite bounds. Exchangeability can fail under time shift, clustering and selection. Nominal marginal coverage is not a guarantee for one trial or a sponsor subgroup.

For the text branch, the 11-row calibration split gives a nominal 90% residual radius of 783.82 days. Observed test coverage is 63/64 (98.44%), with mean clipped interval width 1,440.22 days. This is broad uncertainty, not precise forecasting. The selected baseline has its own interval; do not reuse the text branch's interval in another model's report.

Bootstrap intervals use 200 paired per-trial resamples and seed 42. They condition on fitted models, omit refitting and sponsor/site clustering uncertainty, and do not correct multiple comparisons. Descriptive support warnings check numeric/category/text-length ranges, not validated semantic novelty or a calibrated out-of-distribution probability.

Acceptance check: change evaluation splits and verify row counts/undefined values follow the selected split. Inspect one-class diagnostics and conformal unbounded cases. Do not rank or promote models using the untouched test table.

19. Phase Academy and Phase Model

Phase Academy explains the questions, objectives and common activities associated with clinical research phases. It is an educational aid. Trial phase depends on the intervention and regulatory context, and combined or missing registry labels must remain visible rather than being forced into an invented category.

Use the academy controls to explore stages and inspect actual dataset examples. Play/pause/reset or manual navigation controls illustrate a journey; they do not simulate participant progress, efficacy or regulatory approval. Reduced-motion preference should preserve access to the information without requiring animation.

Phase Model is a separate task from duration and risk. It compares registry phase with a classifier trained on protocol-like inputs. Phase itself is excluded from predictors and direct phase-label terms are masked in text, though indirect textual cues and latest-snapshot leakage can remain.

Output	Meaning
Reported phase	The source record's label
Predicted phase	The saved model's most likely registry-style class
Class probabilities	Uncalibrated model scores; not chances of clinical success
Disagreement	A reason to inspect source and model evidence, not proof the registry is wrong
Unsupported/unknown	Retain the explicit unknown or unavailable state

Prediction Studio can apply the approved phase artifact without retraining. Owners may separately train a phase candidate from an eligible dataset and examine accuracy, balanced accuracy, macro-F1, class support and confusion matrix. Candidate training does not activate it; Model Registry governs approval and activation.

The starter phase test has no EARLY_PHASE1 cases. Never claim measured performance for an absent class. Macro-F1 and balanced accuracy need their recorded class-support conventions to be interpreted correctly. The observed 0.5593 test accuracy is a modest research result.

A human document correction in Protocol Lab preserves the source and review history. It is not a silent overwrite of the registry label or a command to retrain the phase classifier. A regulatory phase decision requires appropriate human expertise beyond this research interface.

Acceptance check: open a reported-phase example in Academy, inspect the same source in Explorer, and run approved phase inference in Prediction Studio. Verify source phase remains unchanged. For a phase experiment, confirm the test confusion matrix totals 59 in the bundled evidence and inspect absent-class support.

20. Configure a real AI provider

AI replies require a real configured provider account, API key and accessible model. The backend supports OpenAI, Anthropic/Claude, Gemini and an OpenAI Chat Completions-compatible HTTPS provider. A configured key does not prove account access, current model availability or scientific superiority. No paid live provider call is claimed as tested in this release.

Put credentials in the **server** `.env` **only**, never in `VITE_` variables, browser settings, datasets or conversations. Stop and restart the server after changing configuration. Use the exact model identifier available to your account; shipped defaults are configuration examples, not live access guarantees.

```
AI_PROVIDER=auto
AI_PROVIDER_PRIORITY=openai,anthropic,gemini,compatible
AI_AUTO_FALLBACK=true
OPENAI_API_KEY=your_private_key
OPENAI_MODEL=your_accessible_model_id
```

Replace placeholders privately. For another provider, set its own `ANTHROPIC_API_KEY`/`ANTHROPIC_MODEL` or `GEMINI_API_KEY`/`GEMINI_MODEL`. Leave unused keys empty. A compatible provider needs `COMPATIBLE_API_KEY`, `COMPATIBLE_BASE_URL` and `COMPATIBLE_MODEL`; the base is an HTTPS API root, often ending in `/v1`, without a `/chat/completions` suffix, credentials, query or fragment.

Auto follows the explicit server priority among configured providers. Eligible error-based fallback can send the bounded research context to another configured provider. Choose a specific provider and review fallback policy if data handling or cost requires tighter control. The order is an operator preference, not a scientifically measured ranking of the best model for research.

Profile	History messages	Context characters	Requested output tokens
Fast	6	9,000	400
Balanced	16	18,000	700
Deep	30	30,000	1,600

Profiles bound retained context and output. They do not secretly choose a better validated model or retrain numerical models. Provider-side policies and limits also apply. The response records actual provider/model and attempted providers.

Acceptance check: diagnostics may report configured provider names, never keys. After you deliberately permit a live test, send a short public-data prompt and verify the actual returned provider/model. Until then, distinguish adapter tests with controlled responses from a real paid-account test.

21. Saved contextual conversations

AI Assistant stores conversations and messages per user within a project. Select the dataset and optional study, create a conversation, then send a clear research question. Examples include asking which source fields are missing, explaining a model warning or summarizing reviewed protocol excerpts. Numerical forecasts must come from identifiable statistical artifacts, not invented chat percentages.

The backend supplies bounded dataset provenance, selected study/PCI details, retained conversation history and relevant project document excerpts. Source references include dataset/document identifiers and hashes. Imported text is treated as untrusted evidence, not instructions to override server safeguards.

1. Create a conversation with a recognizable title.
2. Select the appropriate dataset/study, provider preference and response profile.
3. Ask a bounded question. Review the response text, actual provider/model, references, profile and error state.
4. Ask a follow-up and check the assistant can use relevant prior context.
5. Refresh or restart and reopen the conversation from history. Rename/delete only when you intend that change.

Missing keys return an explicit unavailable/error response. A failed user message is retained as failed and no pretend assistant reply is generated. A retry is another message request, subject to quota. Per-conversation leasing prevents overlapping replies; wait for the current request or its failure before retrying.

Retained context is not unlimited memory. Older turns may remain in storage but be omitted from a provider request to meet profile bounds. Do not assume every earlier document or instruction was included. Source references show supplied context, not independent factual verification of every sentence. Check citations against source text before using a response.

Owners/researchers can create, rename, delete and send messages in their own conversations. Reviewers/viewers cannot call a provider and retain read-only access to their existing history. Another user, including another owner, cannot read your private conversation through these routes.

Retention applies lazily when chat endpoints are accessed, and backups have their own retention. Audit events avoid chat body content, but conversation storage itself contains messages and supplied evidence. Use public/synthetic research data unless a separately reviewed environment and provider policy permits more.

Acceptance check: with controlled provider tests or a deliberately authorized live call, save a two-turn conversation, refresh, restart and reopen. Switch user/project and verify it is not exposed. With no key configured, confirm failure persistence and no fabricated answer.

22. Remote PostgreSQL and durability

For multi-user durable metadata, use managed PostgreSQL such as **Supabase or Neon** with a reviewed persistent server/storage setup. This is an operating recommendation, not a provisioned or live-tested integration. SQLite is the labelled local development fallback; it is not a remote production database.

The app uses SQLAlchemy with psycopg. Core dependencies include the driver. Put the connection string in server `.env`, adapting the scheme to `postgresql+psycopg://` and preserving required provider options. Never paste it into the frontend.

```
DATABASE_URL=postgresql+psycopg://USER:PASSWORD@HOST/DATABASE?sslmode=require
```

Use the exact credentials/endpoint from your provider; URL-encode special characters in credentials. The example requires encrypted transport. For stronger server identity verification, follow your provider's certificate and `verify-full` instructions. Do not weaken TLS to make a failing connection work.

Supabase's official guidance recommends direct connections for persistent backends, with session pooling as an alternative for IPv4-only networks. Neon requires SSL/TLS. Choose a connection mode compatible with this long-running Python process and schema initialization. Provider plans, availability and connection limits require a current account-specific review.

The platform creates private `cti_platform` metadata tables and generated per-project research schemas. Initialization needs schema/table creation privileges. Gateway authorization enforces project membership; this is not a claim of PostgreSQL row-level-security isolation or independent per-tenant credentials.

State	What survives only if backed up
Platform database	Accounts, sessions, projects, memberships, settings, conversations, audit, approvals, active pointers and saved v5 batches
Project registry	Dataset/experiment/scenario metadata in the configured backend
Persistent file storage	Raw files, canonical tables, protocol originals and fitted artifacts

PostgreSQL alone does not preserve file artifacts. Use a private persistent volume and back up database plus files consistently. Changing `DATABASE_URL` does not migrate an existing SQLite workspace automatically. Start a separate test database/storage root or perform a reviewed migration and restore test.

Acceptance check before production: confirm bootstrap reports PostgreSQL; create records, restart, reopen them, and test database plus file restoration. This release has not completed that remote roundtrip or a production deployment. See official connection references in chapter 26.

23. Backup, privacy and operating limits

Back up the complete research state, not just the CSV you imported. For local development, stop the server cleanly before copying the storage root so SQLite metadata and files form a consistent snapshot. Keep the v5 source/release checksum manifest and approved bundled artifacts with the backup. Preserve secrets separately with restricted access.

1. Stop with Ctrl+C and verify the process has exited.
2. Copy the entire configured `STORAGE_DIR`, plus any externally located raw data/artifacts used by your workflow, into a dated protected backup.
3. Retain the exact release source, configurations and dependency records. Store `.env` securely apart from a distributable ZIP.
4. Restore into a separate test folder/storage root and verify datasets, saved batches, model hashes, protocol reviews, conversations and scenarios before trusting the backup.
5. For PostgreSQL, coordinate a database backup with the matching private file-volume snapshot and record restore instructions. Do not assume provider metadata backup includes local model files.

Data is public/synthetic research material unless a separately reviewed environment permits more. Provider requests may transmit bounded dataset text, protocol excerpts and retained chat to the selected provider, and fallback may transmit them to another configured provider. Review consent, contracts, residency and retention requirements before introducing sensitive material.

The app has server role checks, password/session hashing, request/origin limits, bounded import parsing and private storage organization. Those controls do not establish HIPAA, GxP, GDPR or Part 11 compliance. Audit history is application data, not independently immutable storage or a qualified electronic signature system.

Run one server worker. In-process training is not a durable distributed queue. Production needs operator work: HTTPS/reverse proxy, persistent volume, database administration/migrations, backup/restore drills, monitoring, secret management, account recovery, load/security testing and incident handling. Object storage, SSO/MFA, external immutable audit and independent clinical validation are not completed by this release.

Keep `API_HOST=127.0.0.1` for local use. Production rejects local identity mode, absent PostgreSQL and missing TLS configuration. Do not set `APP_ENV=production` and assume the remaining deployment controls have been provided.

Acceptance check: perform a local restore into a different root and verify an existing batch's dataset and artifact hashes. Confirm a distributable package contains neither private `.env` values nor your workbench storage. Never delete the only copy while testing recovery.

24. Troubleshooting and honest failures

Symptom	Meaning and next action
Python launcher cannot find 3.12	Install Python 3.12 with launcher; run `py -3.12 --version`; rerun setup
Missing module or incompatible artifact	Use this release's `.venv`, rerun setup and `pip check`; do not load artifacts under arbitrary global versions
UI build missing	Rebuild `frontend` with `npm ci` and `npm run build`, then restart
Browser cannot connect	Keep server terminal open; use its printed URL; check whether port 8000 is occupied
Port already in use	Stop the previous app or set another `API_PORT` in `.env`, restart, and use that address
Setup token rejected	Use this installation's token; first-owner setup is unavailable after an owner exists
401 / session expired	Sign in again; ensure browser cookies and correct origin
403 / role denial	Request the appropriate project role from an owner; no bearer-token bypass
Import rejected	Read row/field error; check duplicate IDs, numeric values, schema, format and size
Strict batch blocked	Review timing warnings; latest-state data normally lacks valid pretrial feature provenance
Model not trained / no compatible model	Open Model Registry and Prediction Studio; inspect missing/hash/version/schema errors; do not retrain merely because a legacy experiment view is empty
Same duration for many studies	The approved validation-selected median baseline can correctly return a constant
Unknown recruitment/amendment risk	Required validated labels/model are unavailable; this is intentional
AI unavailable / 503	Configure a real server key/model or inspect provider errors; diagnostics do not verify access
AI 429 / quota	Check per-project daily usage, account provider limits and pending conversation request

Ctrl+C, KeyboardInterrupt and CancelledError: if these appear immediately after you intentionally stop Uvicorn, the server may simply be cancelling its lifespan task. Confirm the terminal returns and the process exits. This is different from the same traceback during normal work, repeated startup failure or lost requests. Preserve relevant sanitized logs and run diagnostics for an unexpected failure.

After an interrupted training job, the next server start marks it failed. Review the error and start a new owner run if appropriate. Do not claim a partially downloaded acquisition or incomplete experiment as successful. Preserve genuine partial source files and recovery metadata.

For a persistent issue, record the exact operation, project/dataset IDs, time, sanitized error and diagnostics. Never include passwords, invitation/setup tokens, API keys or database URLs in a support report.

25. Every-screen acceptance and tests

Use a disposable project and public/synthetic inputs. The table is a manual acceptance plan; check each item on the actual running build. Tests with controlled providers or SQLite do not establish paid-provider access, remote PostgreSQL durability or a deployment.

Screen / area	Verify
Login / projects	First claim once; expiry/logout; switch-project isolation
Settings / members	Owner changes, invitation redemption, role denials, quota/retention display
Data Manager	Starter, CSV/XLSX/JSON/Parquet/paste; invalid import creates no dataset
Prediction Studio	Saved artifact inference, strict blocking, export and restart/reopen
Model Registry	Candidate does not activate; approve, activate, rollback; changed artifact fails
Overview / Explorer	Filters, pagination, details, compare and correct exports
Complexity Lab	Reference/custom weighting; missingness; source unchanged
Formula Studio	Six terms, known answers, rounding, manual/reduced-motion controls
Protocol Lab	Source hashes/quotes, version comparison, nullable rubric, reviewed corrections
Research Hub	Self-excluded text retrieval, phase filter, evidence packet identity
Digital Twin	Known-answer arithmetic, persisted scenarios, immutable original
Experiments	Owner-only preflight/train; actual metrics/artifacts; champion unchanged
Validation Lab	Split counts, one-class null metrics, interval and support limitations
Phase Academy / Model	Educational controls; reported label preserved; approved phase inference
AI Assistant	Failed/successful state, actual provider metadata, bounded history, user privacy

For Python regression tests, install test dependencies and run from the root:

```
.venv\Scripts\python.exe -m pip install -r requirements-dev.txt
.venv\Scripts\python.exe -m pytest -q
.venv\Scripts\python.exe -m pip check
cd frontend
npm ci
npm run build
```

Targeted suites include `test_v5_platform.py`, `test_v5_inference.py`, `test_v5_protocol.py`, `test_v5_models.py`, `test_v5_acquisition.py`, provider tests and inherited research/API/phase tests. A passing frontend build verifies compilation, not every browser interaction. Complete the manual checks and inspect runtime errors separately.

Record date, environment, exact command, pass/fail/skip counts and unresolved limits. Optional dependency skips must remain visible. The release integration report contains actual final results; do not replace them with this checklist or a claimed result from a different environment.

26. Reproduction and reference map

Reproduce into new output directories. Do not overwrite approved release artifacts. The frozen source pages support replay; a new live download produces a different changing cohort. CLI fitting produces research artifacts, not automatic project approval or activation.

```
set OMP_NUM_THREADS=1
set OPENBLAS_NUM_THREADS=1
set MKL_NUM_THREADS=1
set NUMEXPR_NUM_THREADS=1
.venv\Scripts\python.exe scripts\download_starter_data.py --replay-from data\starter
  --output-dir data\starter-replayed
.venv\Scripts\python.exe scripts\train_models.py --input data\starter\trials.csv --metadata
  data\starter\metadata.json --config configs\research.json --output
  examples\duration-reproduced
.venv\Scripts\python.exe scripts\train_phase_model.py --input data\starter\trials.csv
  --metadata data\starter\metadata.json --output examples\phase-reproduced
```

Commands are for Command Prompt; copy exact lines from the editable Markdown because PDF lines can wrap visually. On macOS/Linux use `.venv/bin/python`, forward-slash paths and `export` for environment variables.

Optional Cox survival requires curated `nct_id,start_date,followup_date,event_observed`, baseline inputs and endpoint/censoring metadata. Optional BioBERT uses a pinned exact upstream revision and frozen weights, not deep fine-tuning. Neither fit is claimed executed for the approved artifacts. Install optional requirements only for a deliberate research task.

Local reference	Purpose
`V5_MODEL_CARD.md`	Actual cohort, candidates, metrics, uncertainty and acquisition limits
`V5_PRETRAINED.md`	Saved-artifact inference and approval/activation API contract
`V5_PLATFORM.md`	Sessions, roles, persistence, chat, import controls and integration limits
`V5_SCIENTIFIC_BOUNDARIES.md`	Leakage, endpoints, censoring, calibration and interpretation
`V5_REQUIREMENTS_TRACEABILITY.md`	Requirements mapped to implemented, conditional and deferred work
`configs/formulas.json`	Shared exact formula specification and known answers

Official references, checked for the connection guidance in this edition: Supabase PostgreSQL connections, Supabase SSL verification, Neon secure connections, and ClinicalTrials.gov API. Provider connection instructions do not constitute a live test of your account.

To rebuild this PDF from the editable Markdown, install ReportLab in a documentation environment and run `python scripts/build_v5_guide.py`. The generator validates the PCI table against shared configuration and uses real multipass page numbers/bookmarks. Render and inspect the resulting PDF after edits. The shipped v4 guide remains unchanged.